SYNOPSIS

ON

CreatorLens

Submitted in

Partial Fulfillment of the requirements for the Award of the Degree

of

Bachelor of Technology

In

Computer Science and Engineering

by

(Project ID: [PLACEHOLDER — fill in later])

[Student Name — PLACEHOLDER] ([Roll Number — PLACEHOLDER])

Under the supervision of
[Supervisor Name — PLACEHOLDER]
([Supervisor Designation — PLACEHOLDER])



Pranveer Singh Institute of Technology
Kanpur – Agra – Delhi National Highway - 19
Bhauti – Kanpur – 209305

1. Introduction

Content creators on platforms such as YouTube, Instagram and TikTok receive per-post counts of views, likes, comments and shares. Each number is easy to read on its own, but it is difficult to compare many posts that differ in age, platform and format. Older posts keep collecting views, a single viral post distorts an average, and rates calculated in different ways cannot be compared. As a result, creators often judge performance from a recent spike or from intuition instead of from a consistent analysis of their own data.

CreatorLens is a web application developed as a B.Tech Computer Science and Engineering mini project in the creator-analytics domain. It converts a creator's own data into consistent metrics, charts and evidence-labelled insights. One view-based engagement rate, (likes + comments + shares) / views × 100, is used throughout, so that different posts and categories are measured in the same way. Missing counts are never filled in, sample data is always labelled as demo data, and every insight states the evidence behind it.

The implemented system uses a React and TypeScript frontend built with Vite [1], [2], [3] and a FastAPI backend [4]. Metrics are calculated with Pandas and NumPy [5], [6], and results are stored in local SQLite databases [7]. Public YouTube video metadata is read with yt-dlp [8]. Two prediction features are included: an engagement-rate model that compares a mean baseline, a decision tree, a random forest and XGBoost using scikit-learn [9], [10], and a view forecast that uses Holt's linear exponential smoothing [11], [12] only when it performs better than a last-value baseline. Google Gemini [13] is optional and is used only to word summaries; all numbers come from the application's own calculations.

The scope is limited to data that the creator supplies (CSV, PDF and DOCX uploads, and dated view history) and to public YouTube metadata. Private platform analytics such as impressions, click-through rate and audience retention are not available because creator OAuth access is not connected. CreatorLens does not publish or schedule posts, and it does not guarantee growth. Its relevance lies in giving small and early-stage creators a transparent, reproducible way to understand past performance and to estimate future results with stated limitations.

2. Project Objective

The objectives below are stated so that each can be checked against the working application and its automated tests. Table 2.1 separates what is implemented from what is partial or planned.

O1. Dataset upload and validation: accept a creator's post data as a CSV file (and PDF or DOCX documents up to 15 MB), reject unsupported or malformed files with a specific message, and save each successful analysis together with the original file.

O2. Consistent metrics: calculate the view-based engagement rate for every valid post, together with category, platform and weekly-trend summaries and outlier flags, without treating a missing count as zero.

O3. Visual insights: present the results as interactive charts and short insights on the Dashboard, Analytics, Insights and saved-analysis pages, each with the evidence and limitations behind it.

O4. YouTube video analysis: analyse a public YouTube video from its URL (views, likes, comments, duration, engagement rate, views per day) and generate an evidence-based performance diagnosis with an action plan.

O5. Engagement-rate prediction: predict the engagement rate of a planned post from fields known before publication, select the best of four models on held-out posts, and explain which inputs influenced the result.

O6. View forecasting: forecast a video's views after 24 hours, 3 days and 7 days from dated observations, using Holt's model only when back-testing shows that it is at least 10% more accurate than a last-value baseline.

O7. Creator-specific records: keep saved analyses associated with an account through server-side ownership checks (implemented in the backend; sign-in is currently paused, as noted in Table 2.1).

Table 2.1: *Implementation status of CreatorLens features (verified from the repository on 21 Oct 2026)*

Feature	Status	Evidence in repository
Demo dashboard, analytics and insights (16 labelled demo posts)	Implemented	demo_dataset.py; /api/dashboard/*

Feature	Status	Evidence in repository
CSV/PDF/DOCX upload, validation, saved analysis and questions	Implemented	document_analysis.py; /api/documents
YouTube public-video analysis and AI Performance Diagnosis	Implemented	youtube_analysis.py; diagnosis.py
Roast My Video and Content Court (creative feedback)	Implemented	roast.py
Creator analytics CSV (dated rows) analysis	Implemented	analytics_analysis.py
Engagement-rate prediction model	Implemented; evaluated only on a synthetic test dataset so far	engagement_model.py; /api/predictions/*
View forecasting (Holt vs. last-value baseline)	Implemented; saved videos currently have one observation each, so no real forecast yet	prediction.py; view_history.py
Accounts: registration, email verification, login, password reset	Partially implemented: backend complete, sign-in paused; email needs SMTP	auth.py; auth_routes.py
Gemini-written summaries and wording	Optional: needs API key and quota; calculated results remain without it	diagnosis.py; document_analysis.py
Gemini video-file analysis	Partial: backend endpoint only, no page	video_routes.py (/api/videos/analyze)
Private YouTube Analytics (impressions, CTR, retention)	Planned: OAuth not connected	youtube_client.py
Automatic scheduled view snapshots	Planned: manual or command-line collection only	view_history.py --collect
PostgreSQL and Clerk authentication	Not implemented (planning documents only)	TRD.md; architecture.md

Expected benefit to the end user: a creator can upload existing data and immediately see which content types and platforms earn more engagement per view, understand why a video looks strong or weak from the evidence available, and obtain a prediction or forecast that is accompanied by its measured error and limitations. This reduces manual spreadsheet work and discourages decisions based on a single post.

3. Feasibility Study

A feasibility study examines whether the project can succeed by assessing its technical, operational, economic, schedule and legal aspects. Each aspect is assessed below against the system that has actually been built.

3.1 Technical Feasibility

All core technologies are mature and open source: React, TypeScript and Vite for the interface [1], [2], [3], Recharts for charts [14], Tailwind CSS for styling [15], FastAPI for the REST API [4], and Pandas, NumPy, scikit-learn, XGBoost and statsmodels for analysis and modelling [5], [6], [9], [10], [12]. The datasets are small and tabular, so training and forecasting run on an ordinary laptop CPU without a GPU. SQLite [7] removes the need for a separate database server. The backend has 142 automated pytest tests [16], all of which passed on 21 October 2026, and the frontend passes a TypeScript type check and a production build.

Two external dependencies carry technical risk. First, yt-dlp [8] depends on YouTube's public page structure and can be blocked by bot checks; the backend reports such failures as clear errors instead of inventing counts. Second, the optional Gemini API [13] is limited by quota; during development the free-tier daily request limit was reached, and the application continued with its calculated results. The project is therefore technically feasible with these documented limitations.

3.2 Operational Feasibility

A creator uses CreatorLens in a browser without writing code. The implemented workflow is: open the Dashboard to explore labelled demo data; upload a posts CSV on the Upload Dataset page and read the calculated summary, charts and outliers; paste a public YouTube URL on the Analyze YouTube page to obtain statistics, a diagnosis and optional creative feedback; and open the Predictions page to train the engagement-rate model on a saved posts CSV or to request a view forecast for a saved video. Every page explains when data is insufficient, for example that one public snapshot cannot establish a trend, and tells the creator what to collect next. The main operational limitation is that view forecasts need dated observations collected over time or imported from a history CSV.

3.3 Economic Feasibility

Development uses free and open-source software on an existing computer, so no licence or server cost has been incurred. Table 3.1 summarises the cost position. Gemini is the only component that may require payment beyond its free usage limits; it is optional, and its cost was not estimated for this synopsis. Hosting costs will arise only if the application is deployed publicly in future.

Table 3.1: *Cost assessment*

Item	Cost for this project	Remarks
Frontend, backend and ML libraries	Free (open source)	React, FastAPI, Pandas, scikit-learn, XGBoost, statsmodels
Database	Free	SQLite files on the local disk; no database server
Development tools	Free	Python, Node.js, npm, Git, code editor
Google Gemini API	Free tier with limits; paid use is usage-based	Optional; the app works without it
Public YouTube metadata	Free	Read with yt-dlp; no paid API key
Deployment and hosting	Not incurred	Future cost depends on the chosen provider

3.4 Schedule Feasibility

Git commits and file timestamps show that the planning documents were written on 14 October 2026 and that the implemented modules were developed between 14 and 20 October 2026. The remaining work, namely integration testing, re-enabling sign-in, evaluation on real creator data, and the final report, is shown as a proposed plan with estimated dates in the Gantt chart (Figure 4.4 and Table 4.1 in Section 4.4). The project is feasible within the mini-project schedule because the core features already work and the remaining tasks are testing, evaluation and documentation.

3.5 Legal and Ethical Feasibility

CreatorLens processes data that creators upload about their own content. Uploaded files and databases are kept in the local backend/data folder, which is excluded from version control. Account passwords are stored as PBKDF2-SHA256 hashes with 600,000 iterations [17], [18], sessions use HttpOnly cookies, and state-changing requests require a CSRF token. Because the application handles personal data such as names and email addresses, any public deployment in India would need to consider the Digital Personal Data Protection Act, 2023 [19]; compliance has not been assessed and is not claimed.

Public YouTube metadata is read with yt-dlp for academic demonstration. YouTube's Terms of Service [20] restrict automated access to the service, so a production version should use YouTube's official APIs with creator consent. When a Gemini key is configured, extracted document text and calculated evidence are sent to Google [13], and users should be informed of this. Ethically, the

system never fabricates metrics, labels demo data, presents predictions as estimates rather than guarantees, describes feature importance as association rather than cause, and keeps creative feedback focused on content instead of the person.

4. Methodology / Planning of Work

The following steps describe how CreatorLens processes data from input to result. Each step reflects the code in the repository; partial or planned elements are identified explicitly.

Step 1. User access. The creator opens the React application in a browser. Account registration, email verification, login, logout and password reset are implemented in the backend, but sign-in is currently paused, so all saved work is stored under a single local account.

Step 2. Data input. The creator can upload a CSV, PDF or DOCX file (up to 15 MB) on the Upload Dataset page, paste a public YouTube URL, import a dated view-history CSV with the columns observed_at and views, or explore the built-in demo dataset of 16 labelled posts.

Step 3. Validation and preprocessing. The backend checks file extension, content type and size; parses CSV as UTF-8 within row and column limits; converts numeric fields; drops invalid rows with a stated reason; converts timestamps to UTC and rejects duplicates and future dates. PDF and DOCX text is extracted with pypdf and python-docx. A YouTube URL is reduced to its video ID, and public metadata is cached for six hours.

Step 4. Analysis and metric calculation. Pandas computes the engagement rate for each valid post, category and platform averages, weekly trends, and outliers outside 1.5 times the interquartile range. For a YouTube video, views per day and the comment rate are calculated, and a creator median is used as a baseline only when at least three other videos from the same channel have been saved.

Step 5. Visualisation and insight generation. Recharts draws trend, comparison and interaction charts. The performance diagnosis is written from rule-based evidence and labelled Limited, Moderate or Stronger evidence. Gemini may reword it only if every cited figure still matches the calculation.

Step 6. Prediction and forecasting. The engagement-rate model and the view forecast are run on request from the Predictions page. Both are described after these steps.

Step 7. Storage and retrieval. Analyses are saved as JSON payloads in SQLite tables with an owner column; uploaded files are kept in backend/data/files and trained models in backend/data/models (Figure 4.3).

Step 8. Display of results. The frontend shows the results on the Dashboard, Analytics, Insights, Saved analyses, YouTube analysis and Predictions pages, including loading states and clear error messages.

Engagement-Rate Prediction Model (Step 6)

The target is the engagement rate of a post. To avoid target leakage, the model uses only fields that are known before publication: platform, category, posting date (weekday and month), posting hour, title (length, word count, digits, question mark) and duration. Views, likes, comments, shares and other outcome columns are excluded from the inputs. A saved CSV must contain at least 30 usable posts with views above zero.

The posts are sorted by date, and the latest 20% (at least six posts) are held out for testing. Four scikit-learn pipelines are trained on the earlier posts: a mean baseline, a decision tree [21], a random forest [22] and XGBoost [10]. The model with the lowest mean absolute error (MAE) on the held-out posts is selected and refitted on all posts; if no model beats the mean baseline, predictions are refused. Input importance is measured by shuffling each input on the held-out posts, and when XGBoost is selected, per-prediction TreeSHAP contributions [23] are shown. On a 200-post synthetic test file, XGBoost achieved an MAE of 0.84 percentage points against 0.92 for the mean baseline, 0.89 for the random forest and 1.03 for the decision tree. These figures come from test data and are not evidence of accuracy on real creator data.

View Forecasting with Holt's Linear Exponential Smoothing (Step 6)

A video's cumulative view count is forecast from dated observations of the same video. Holt's method [11], [24] maintains a smoothed level ℓ and trend b, which are updated at each observation y:

$$\ell_t = \alpha \cdot y_t + (1 - \alpha)(\ell_{t-1} + b_{t-1})$$

$$b_t = \beta(\ell_t - \ell_{t-1}) + (1 - \beta) \cdot b_{t-1}, \qquad \hat{y}_{t+h} = \ell_t + h \cdot b_t$$

Here α and β are smoothing parameters estimated by statsmodels [12], and h is the number of steps ahead. Because real snapshots are taken at irregular times, the views are first placed on a regular

1-, 3-, 6-, 12- or 24-hour grid by linear interpolation between real observations. The baseline predicts that future views equal the latest count, $\hat{y} = y_t$.

With fewer than three observations, no forecast is produced. With three to five, a provisional baseline forecast is shown. With six or more, a rolling-origin back-test [25] is run: from each earlier origin, both methods predict the real observations that follow within seven days, and Holt's model is selected only if its MAE is at least 10% lower than the baseline's. Forecasts for 24 hours, 3 days and 7 days are never below the latest count, and any horizon longer than twice the recorded history is hidden. Saved videos currently have one observation each, so the application shows guidance to collect snapshots instead of a forecast.

4.1 System Architecture Diagram

Figure 4.1 shows the verified architecture. The browser runs the React single-page application, which calls the FastAPI backend through the /api path. API routers delegate work to service modules for data processing, document extraction, diagnosis, the engagement model and view forecasting. Services read and write SQLite databases and local folders, read public YouTube metadata with yt-dlp, and optionally call the Gemini API. Figures 4.1 to 4.4 are placed on landscape pages at the end of this section so that their labels remain legible.

4.2 Data Flow Diagram

Figure 4.2 is a level-1 data flow diagram. The creator supplies files, URLs, planned-post details and history CSVs to five processes. The processes exchange data with four data stores and with the two external entities, YouTube public pages and the optional Gemini API, and they return metrics, diagnoses, predictions and forecasts to the creator.

4.3 ER Diagram

An ER diagram was chosen because CreatorLens persists its data in a relational SQLite schema. Figure 4.3 shows the ten tables defined in the code, grouped by database file. Enforced foreign keys link sessions and email tokens to users and questions to documents. The user_id columns of the analysis tables and the video_id links between YouTube tables are logical links without foreign-key constraints, as implemented.

4.4 Project Timeline and Gantt Chart

Figure 4.4 shows the project timeline in days. Completed phases are dated from Git commits (15 and 18 October 2026) and from file creation and modification times (14 to 20 October 2026). Work after the synopsis date of 21 October 2026 is a proposed plan; its dates are estimates and will be confirmed with the supervisor. Table 4.1 lists the same phases with their status and evidence.

Table 4.1: *Project phases, dates and status*

Phase	Dates (2026)	Status	Evidence
Requirement analysis and problem definition	14 Oct	Completed	PRD.md, TRD.md
System design and architecture	14 Oct	Completed	architecture.md
Frontend shell, demo dashboard, FastAPI foundation	14–15 Oct	Completed	Commit 5a4dc11
CSV validation and document analysis	15 Oct	Completed	Commit d3d28ec
YouTube public-metadata analysis	15–16 Oct	Completed	Commit d3d28ec; youtube_client.py
Creator analytics CSV and diagnosis	16 Oct	Completed	analytics_csv.py; diagnosis.py
Accounts and SQLite data ownership	17 Oct	Completed; sign-in now paused	auth.py; commit c9aca1b
Roast My Video	17 Oct	Completed	roast.py; commit c9aca1b
Engagement-rate model	18 Oct	Completed (not yet committed)	engagement_model.py
View forecasting	19–20 Oct	Completed (not yet committed)	prediction.py; view_history.py
Automated backend tests	14–20 Oct	Completed (142 tests)	backend/tests
Literature review and synopsis	21 Oct	In progress	This document
Integration testing and debugging	22–31 Oct (est.)	Proposed	—
Re-enable sign-in, verify per-user isolation	27 Oct–5 Nov (est.)	Proposed	—
Collect real creator data and evaluate models	27 Oct–9 Nov (est.)	Proposed	—
Final report, presentation and submission	6–14 Nov (est.)	Proposed	—

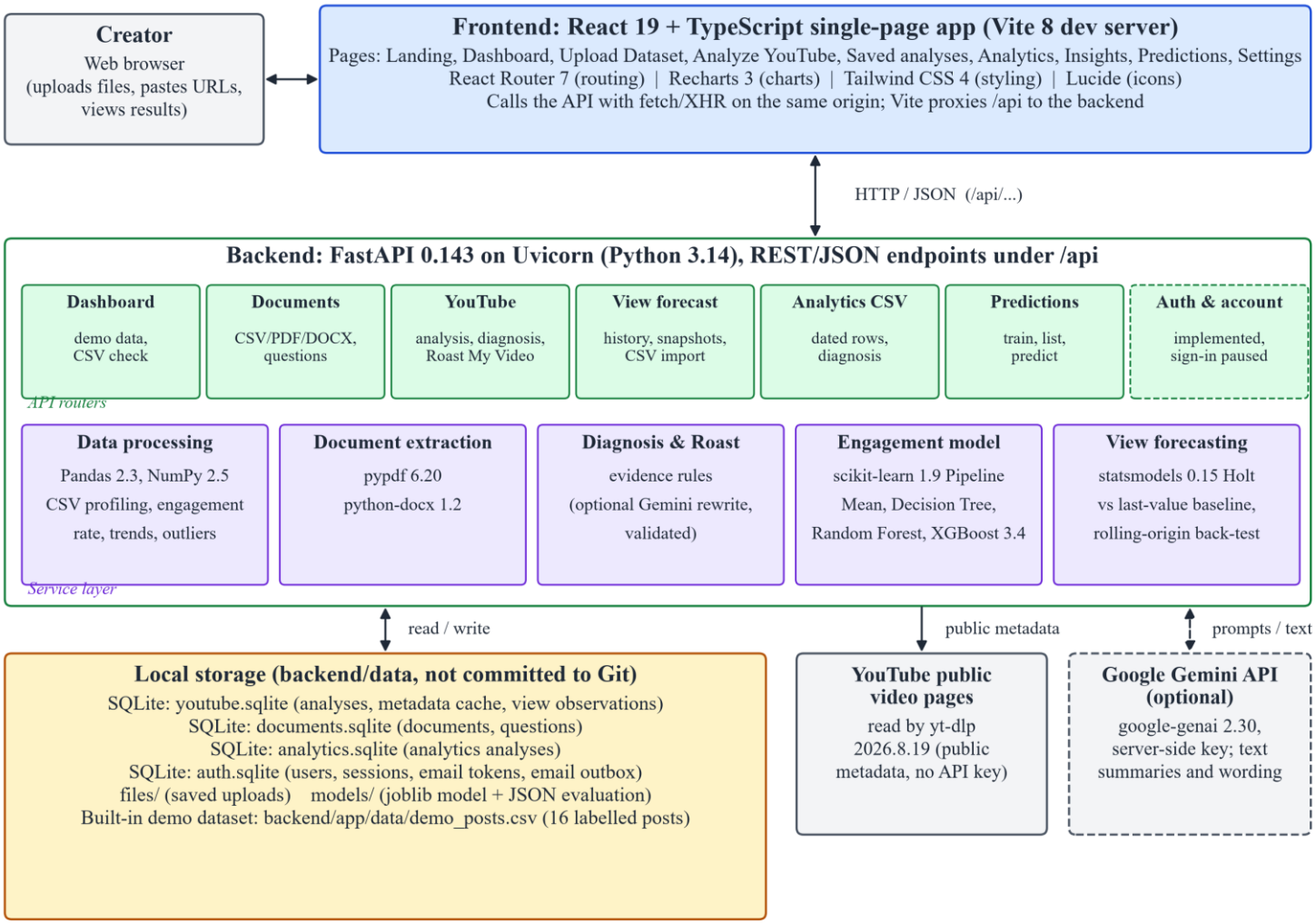


Figure 4.1: *System architecture of CreatorLens (implemented components)*

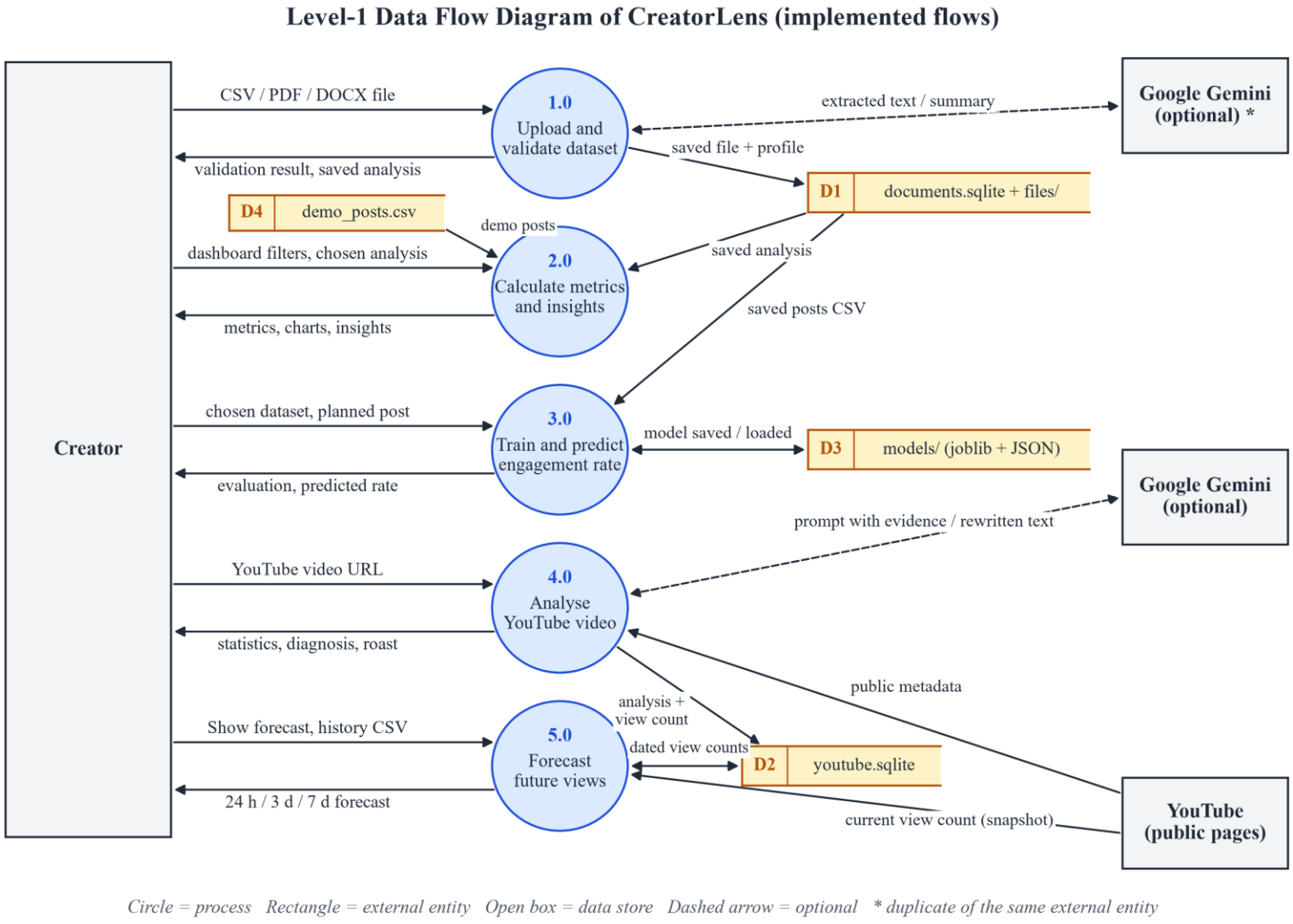


Figure 4.2: *Level-1 data flow diagram of CreatorLens*

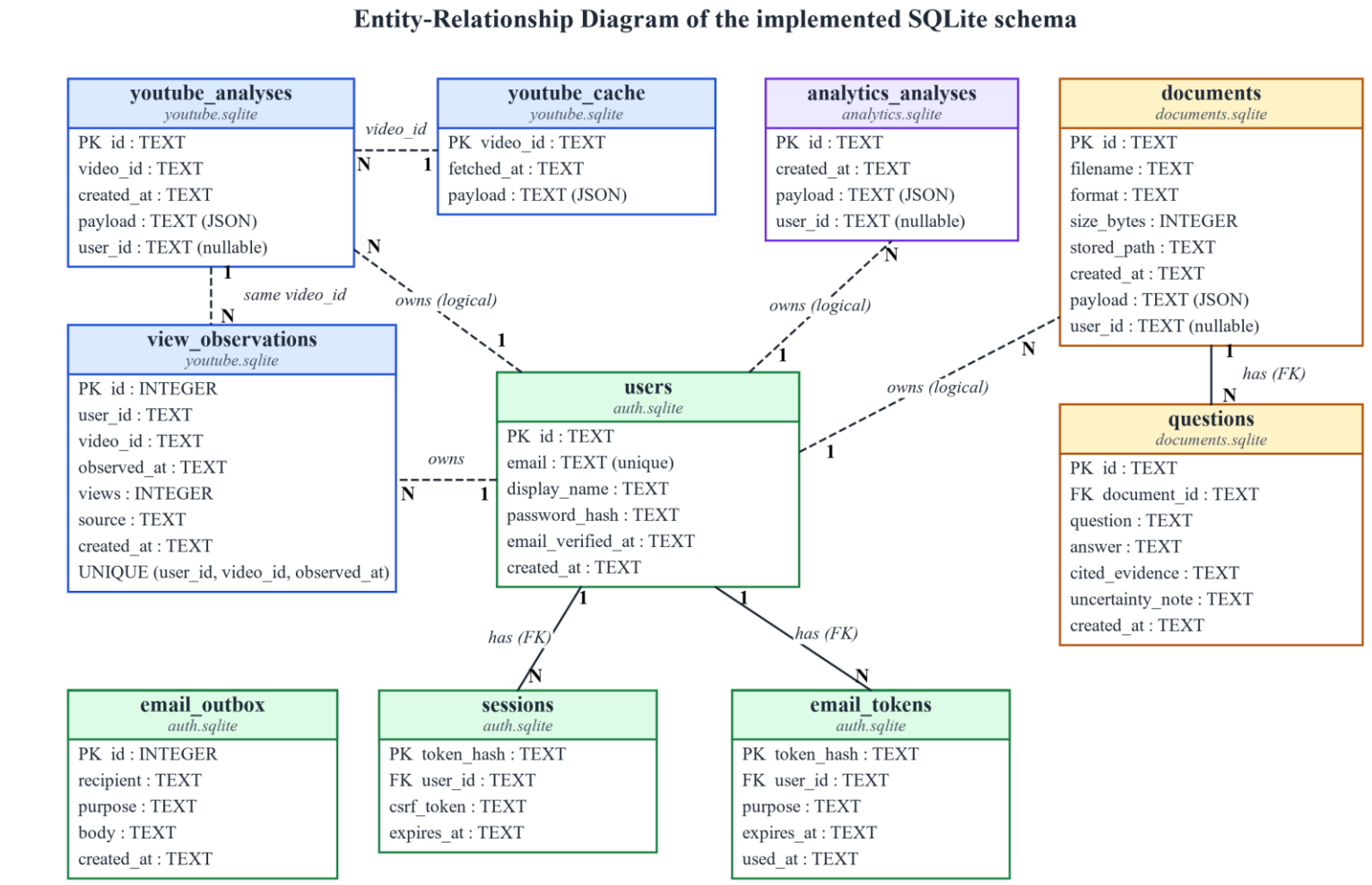


Figure 4.3: *Entity-relationship diagram of the implemented SQLite schema*

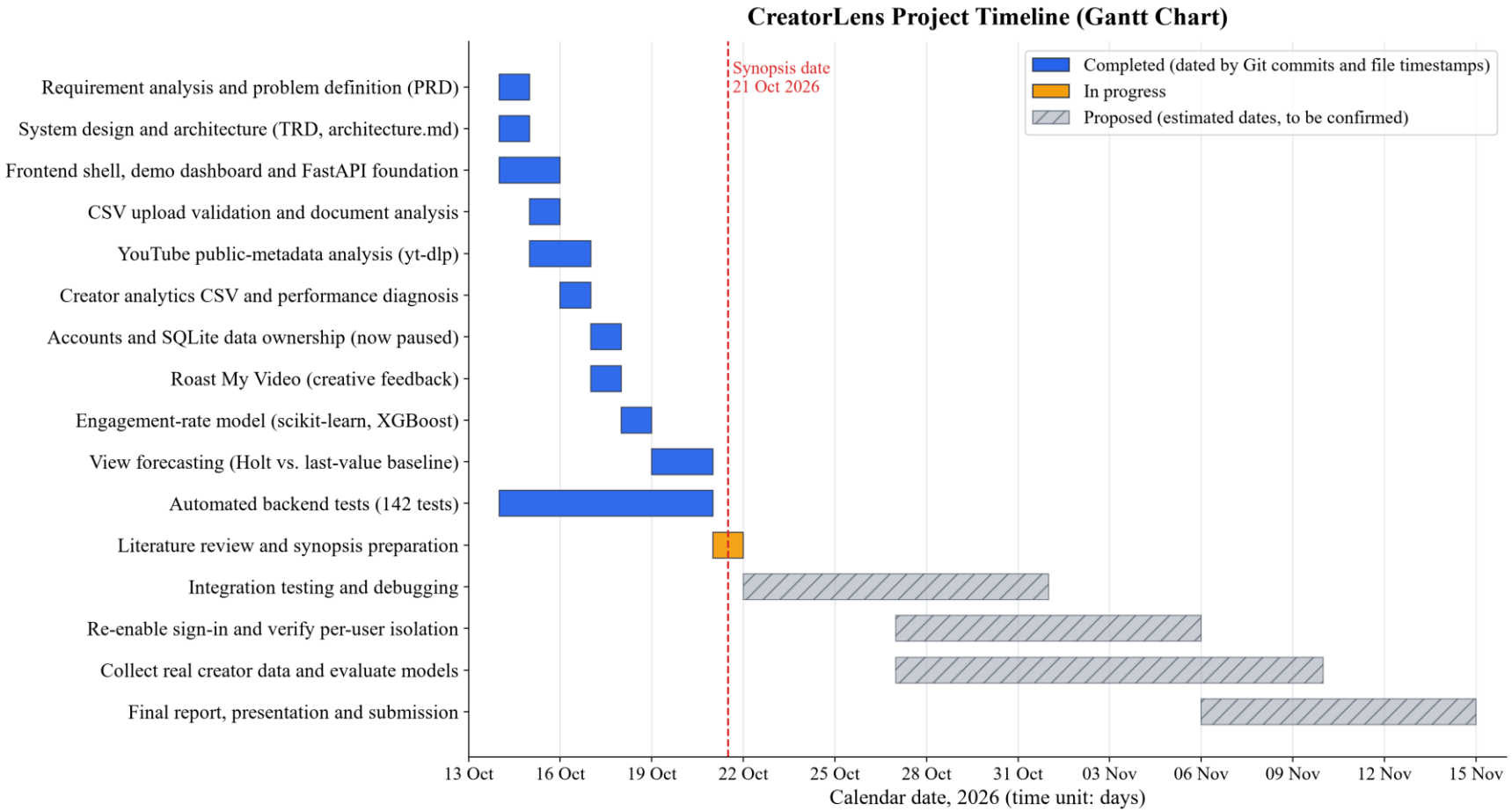


Figure 4.4: *Gantt chart of the CreatorLens project timeline*

5. Tools/Technology Used

5.1 Minimum Hardware Requirements

The hardware below is required for developing and running the project locally. The specifications are estimated minimums; only the disk usage figures were measured in the repository. The exact hardware of the development computer was not recorded in the project files.

Table 5.1: *Minimum hardware requirements (estimated)*

Component	Minimum requirement	Remarks
CPU	64-bit dual-core x86-64 processor	Quad-core recommended for running the frontend, backend and browser together
RAM	4 GB (8 GB recommended)	Model training uses small tabular files in memory
GPU	Not required	XGBoost, scikit-learn and statsmodels run on the CPU in this project
HDD/SSD	At least 3 GB free; SSD recommended	Measured: frontend node_modules ≈ 174 MB, backend virtual environment ≈ 527 MB
Others	Internet connection; modern web browser	Needed for installing packages, reading YouTube metadata and optional Gemini calls

5.2 Minimum Software Requirements

Table 5.2 lists the software verified in the repository's dependency files, installed packages and source code. Versions are those installed in the development environment on 21 October 2026.

Table 5.2: *Software and technologies used (verified)*

Category	Technology and version	Role in CreatorLens
OS	Windows 11 Home (development)	Development and testing environment
Runtime	Python 3.14.4; Node.js 24.15.0 with npm 11.12.1	Backend runtime; frontend tooling
Frontend	React 19.3.1, React DOM 19.3.1, TypeScript 6.0.3	User interface and type safety
Build tool	Vite 8.3.4 with @vitejs/plugin-react 6.1.2	Development server, /api proxy and production build
Routing and UI	React Router DOM 7.18.4; Tailwind CSS 4.3.3; Lucide React 1.57.0	Page routing, styling and icons
Charts	Recharts 3.10.1	Trend, comparison, prediction and forecast charts
Backend	FastAPI 0.143.1; Uvicorn 0.54.0; Pydantic 2.14.1; python-multipart 0.0.32	REST API, ASGI server, request and response validation, file uploads

Category	Technology and version	Role in CreatorLens
Data analysis	Pandas 2.3.3; NumPy 2.5.4	CSV parsing, metrics, trends and outliers
Machine learning	scikit-learn 1.9.1; XGBoost 3.4.1; joblib 1.6.0	Engagement-rate model pipelines, model comparison and model storage
Time-series forecasting	statsmodels 0.15.0 (Holt's linear exponential smoothing)	View forecasting with back-testing against a last-value baseline
Database	SQLite 3.50.4 (Python sqlite3 module)	youtube, documents, analytics and auth databases
Document extraction	pypdf 6.20.1; python-docx 1.2.0	Text extraction from PDF and DOCX uploads
External data	yt-dlp 2026.8.19	Reading public YouTube video metadata
AI service (optional)	google-genai 2.30.1; default model gemini-3.8-flash	Server-side summaries and wording when a key is configured
Authentication	Custom: PBKDF2-SHA256, HttpOnly session cookie, CSRF token	Implemented in the backend; sign-in currently paused
Testing and quality	pytest 8.4.2; httpx 0.28.1; oxlint 1.88.0	142 backend tests; API test client; frontend lint
Development tools	Git; Cursor code editor; web browser	Version control, editing and manual testing

Planned but not used: the planning documents propose PostgreSQL with SQLAlchemy and Alembic, and Clerk authentication. None of these is used in the current code; the implementation uses SQLite and its own session-based authentication.

6. References

[1] Meta Open Source, "React documentation." [Online]. Available: https://react.dev/ (accessed Oct. 21, 2026).

[2] Microsoft, "TypeScript documentation." [Online]. Available: https://www.typescriptlang.org/docs/ (accessed Oct. 21, 2026).

[3] Vite, "Vite guide." [Online]. Available: https://vite.dev/guide/ (accessed Oct. 21, 2026).

[4] S. Ramírez, "FastAPI documentation." [Online]. Available: https://fastapi.tiangolo.com/ (accessed Oct. 21, 2026).

[5] W. McKinney, "Data structures for statistical computing in Python," in Proc. 9th Python in Science Conf. (SciPy 2010), Austin, TX, USA, 2010, pp. 56–61, doi: 10.25080/Majora-92bf1922-00a.

[6] C. R. Harris et al., "Array programming with NumPy," Nature, vol. 585, no. 7825, pp. 357–362, Sep. 2020, doi: 10.1038/s41586-020-2649-2.

[7] SQLite, "SQLite documentation." [Online]. Available: https://www.sqlite.org/docs.html (accessed Oct. 21, 2026).

[8] yt-dlp contributors, "yt-dlp: A feature-rich command-line audio/video downloader," GitHub repository. [Online]. Available: https://github.com/yt-dlp/yt-dlp (accessed Oct. 21, 2026).

[9] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[10] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD '16), San Francisco, CA, USA, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[11] C. C. Holt, "Forecasting seasonals and trends by exponentially weighted moving averages," Int. J. Forecasting, vol. 20, no. 1, pp. 5–10, 2004 (reprint of 1957 ONR memorandum), doi: 10.1016/j.ijforecast.2003.09.015.

[12] S. Seabold and J. Perktold, "Statsmodels: Econometric and statistical modeling with Python," in Proc. 9th Python in Science Conf. (SciPy 2010), Austin, TX, USA, 2010, pp. 92–96, doi: 10.25080/Majora-92bf1922-011.

[13] Google, "Gemini API documentation." [Online]. Available: https://ai.google.dev/gemini-api/docs (accessed Oct. 21, 2026).

[14] Recharts, "Recharts: A composable charting library built on React components." [Online]. Available: https://recharts.org/ (accessed Oct. 21, 2026).

[15] Tailwind Labs, "Tailwind CSS documentation." [Online]. Available: https://tailwindcss.com/docs (accessed Oct. 21, 2026).

[16] pytest-dev, "pytest documentation." [Online]. Available: https://docs.pytest.org/ (accessed Oct. 21, 2026).

[17] K. Moriarty, B. Kaliski, and A. Rusch, "PKCS #5: Password-based cryptography specification version 2.1," Internet Engineering Task Force, RFC 8018, Jan. 2017, doi: 10.17487/RFC8018.

[18] OWASP Foundation, "Password storage cheat sheet," OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (accessed Oct. 21, 2026).

[19] Government of India, "The Digital Personal Data Protection Act, 2023," Act No. 22 of 2023, The Gazette of India, Aug. 2023.

[20] YouTube, "Terms of Service." [Online]. Available: https://www.youtube.com/t/terms (accessed Oct. 21, 2026).

[21] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, Classification and Regression Trees. Belmont, CA, USA: Wadsworth, 1984.

[22] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5–32, Oct. 2001, doi: 10.1023/A:1010933404324.

[23] S. M. Lundberg et al., "From local explanations to global understanding with explainable AI for trees," Nature Machine Intelligence, vol. 2, no. 1, pp. 56–67, Jan. 2020, doi: 10.1038/s42256-019-0138-9.

[24] R. J. Hyndman and G. Athanasopoulos, Forecasting: Principles and Practice, 3rd ed. Melbourne, Australia: OTexts, 2021. [Online]. Available: https://otexts.com/fpp3/ (accessed Oct. 21, 2026).

[25] L. J. Tashman, "Out-of-sample tests of forecasting accuracy: An analysis and review," Int. J. Forecasting, vol. 16, no. 4, pp. 437–450, Oct. 2000, doi: 10.1016/S0169-2070(00)00065-0.